

Berdasarkan riset akademik yang mendalam menggunakan data dari arXiv dan Google Scholar, fine-tuning **T5Gemma-2** untuk multi-turn chatbot multi-task implicit adalah pendekatan yang sangat feasible dan didukung oleh literatur ilmiah yang kuat. Arsitektur encoder-decoder T5Gemma-2, dengan kemampuan bidirectional encoding dan cross-attention, memberikan keunggulan fundamental dibandingkan decoder-only models untuk deteksi intent implisit dalam konteks percakapan. Paper T5Gemma-2 (Zhang et al., 2025) menunjukkan bahwa model ini secara konsisten mengungguli Gemma 3 pada post-training benchmarks, bahkan hanya dengan minimal supervised fine-tuning. Ditambah dengan FLAN Collection yang mencakup **1.945 task** di berbagai domain (Longpre et al., 2023), Anda memiliki fondasi task prefix yang sangat kaya untuk melatih model mendeteksi intent tanpa prefix eksplisit. Daftar prefix yang dapat Anda terapkan mencakup format klasik T5 seperti `summarize:`, `translate English to German:`, hingga instruksi natural language dari FLAN seperti `Answer the question based on the context:`, `Extract all named entities from the following text:`, dan `Rewrite the following sentence in a simpler way:`. Kunci keberhasilannya terletak pada penggunaan **LoRA** untuk efisiensi, **mixed training data** yang menggabungkan format dengan dan tanpa prefix, serta pemanfaatan **conversation history** sebagai signal tambahan untuk deteksi intent implisit.

Laporan Riset Akademik: Fine-Tuning T5Gemma-2 untuk Multi-Turn Chatbot Multi-Task Implicit

Bab ini menyajikan temuan riset akademik yang mendalam terkait rencana fine-tuning model T5Gemma-2 untuk multi-turn chatbot dengan kemampuan multi-task implicit. Riset ini dilakukan dengan mengakses dan menganalisis paper-paper akademik dari berbagai sumber kredibel, termasuk arXiv, Google Scholar, dan repositori resmi Google Research. Fokus utama dari riset ini adalah untuk mengidentifikasi landasan teoretis, strategi task prefix, dan framework implementasi yang didukung oleh literatur ilmiah terkini. Analisis mencakup studi mendalam terhadap arsitektur T5Gemma-2, koleksi task dari FLAN (Fine-tuned Language Net), serta berbagai pendekatan untuk implicit task detection pada model encoder-decoder. Dengan menggabungkan temuan dari berbagai paper, bab ini bertujuan untuk memberikan rekomendasi berbasis bukti yang kuat untuk pengembangan chatbot multi-task yang mampu beroperasi tanpa instruksi eksplisit dari pengguna. Seluruh analisis didasarkan pada data akademik yang sah, dengan referensi yang jelas terhadap sumber aslinya untuk memastikan validitas dan reproduisibilitas temuan.

1. Landasan Teoretis dari Paper Akademik

Bagian ini membahas fondasi ilmiah yang mendukung penggunaan model T5Gemma-2 untuk tugas multi-turn dan multi-task. Analisis berfokus pada kekuatan arsitektural encoder-decoder, efektivitas framework text-to-text, dan metodologi instruction tuning yang telah terbukti secara empiris dalam berbagai paper penelitian. Pembahasan dimulai dengan pemaparan detail arsitektur T5Gemma-2 berdasarkan paper resminya, yang menjelaskan inovasi seperti tied embeddings dan merged attention. Selanjutnya, dieksplorasi keunggulan inheren arsitektur encoder-decoder dalam memahami konteks percakapan panjang dan mendeteksi intent pengguna secara implisit melalui mekanisme bidirectional attention dan cross-attention. Terakhir, bagian ini menguraikan konsep instruction tuning yang dipopulerkan oleh FLAN dan T0, yang menunjukkan bagaimana pelatihan pada beragam instruksi natural language dapat menghasilkan model yang sangat adaptif dan mampu menggeneralisasi ke tugas-tugas baru tanpa memerlukan fine-tuning spesifik.

1.1 Arsitektur dan Keunggulan T5Gemma-2

Model T5Gemma-2 merepresentasikan evolusi signifikan dalam keluarga model encoder-decoder, dibangun di atas fondasi Gemma 3 dari Google DeepMind. Paper “T5Gemma 2: Seeing, Reading, and Understanding Longer” oleh Zhang et al. (2025) secara komprehensif merinci arsitektur dan metodologi pelatihan model ini, yang secara kolektif berkontribusi pada kemampuannya yang superior dalam pemrosesan bahasa alami, terutama dalam konteks multi-turn dan multi-task [1]. Berbeda dengan pendekatan desain model dari nol, T5Gemma-2 memanfaatkan strategi adaptasi yang cerdas dengan mengonversi model decoder-only yang telah terprestasi (Gemma 3) menjadi arsitektur encoder-decoder melalui proses yang dikenal sebagai adaptation via UL2 (Unifying Language Learning Paradigms). Proses ini tidak hanya mempertahankan pengetahuan yang telah dipelajari oleh Gemma 3 selama pre-training, tetapi juga memperkenalkan keunggulan mekanis dari arsitektur encoder-decoder, yang sangat penting untuk tugas-tugas yang memerlukan pemahaman mendalam terhadap input sebelum menghasilkan output, seperti summarization, question answering, dan terutama, percakapan multi-turn. Ketersediaan model dalam tiga ukuran—270M-270M, 1B-1B, dan 4B-4B—memberikan fleksibilitas bagi peneliti dan praktisi untuk menyeimbangkan antara kebutuhan komputasional dan tingkat kompleksitas tugas yang dihadapi [2].

1.1.1 Inovasi Arsitektur: Tied Embeddings dan Merged Attention Dua inovasi utama yang diperkenalkan dalam T5Gemma-2 adalah “**Tied Embeddings**” dan “**Merged Attention**”, keduanya dirancang untuk meningkatkan efisiensi parameter dan komputasional tanpa mengorbankan kinerja secara signifikan. Konsep **Tied Embeddings** berarti seluruh word embeddings—baik yang digunakan di input encoder, input decoder, maupun output decoder (lapisan softmax)—dibagikan dan memiliki parameter yang sama. Dalam arsitektur tradisional, embeddings untuk ketiga bagian ini seringkali terpisah, yang secara substansial meningkatkan jumlah total parameter, terutama untuk model dengan vocabulary yang besar. Dengan men-tie embeddings, T5Gemma-2 berhasil mengurangi jumlah parameter hingga **10.5%** dibandingkan jika embeddings tersebut dipisahkan [2]. Pengurangan ini tidak hanya menghemat memori selama pelatihan dan inferensi tetapi juga mempercepat proses komputasi. Lebih penting lagi, paper menunjukkan bahwa pengurangan parameter ini hampir tidak berdampak negatif pada kualitas output model, yang menunjukkan adanya redundansi tinggi dalam representasi embeddings yang terpisah. Efisiensi ini memungkinkan model untuk mencapai kinerja yang sebanding dengan model yang lebih besar atau, dalam ukuran yang sama, beroperasi dengan lebih cepat dan hemat sumber daya.

Inovasi kedua, **Merged Attention**, bertujuan untuk menyederhanakan struktur decoder pada arsitektur encoder-decoder. Secara tradisional, setiap blok dalam decoder Transformer terdiri dari tiga sub-lapisan: self-attention (yang bersifat causal/masked), cross-attention (yang menghubungkan decoder dengan output encoder), dan feed-forward network. T5Gemma-2 mengusulkan untuk menggabungkan dua mekanisme attention ini—self-attention dan cross-attention—menjadi satu modul attention yang bersifat *unified*. Ini dicapai dengan menggabungkan (concatenate) input decoder (X) dan output encoder (H), dan kemudian menerapkan mekanisme attention tunggal pada gabungan tersebut. Mekanisme masking (M) yang cermat dirancang untuk memastikan bahwa decoder tetap hanya dapat mengakses token-token sebelumnya dalam urutan target (sifat autoregresif) sambil juga dapat mengakses seluruh representasi dari input sumber melalui output encoder. Implementasi Merged Attention ini berhasil menghemat parameter sebesar **6.5%** [2]. Meskipun terdapat sedikit penurunan kualitas sekitar 0.3 poin pada evaluasi pre-training, penulis paper menganggap kompromi ini sangat dapat diterima mengingat penghematan parameter dan komputasi yang signifikan. Inovasi ini tidak hanya meningkatkan efisiensi tetapi juga menyederhanakan arsitektur decoder, menjadikannya lebih mirip dengan arsitektur decoder-only, yang

memfasilitasi proses inisialisasi parameter dari model Gemma 3 decoder-only.

Konfigurasi Arsitektur (2B-2B)	Pre-training Performance	Total #Parameters (Embedding Parameters)
Baseline (Pisah)	47.8	4417M (1180M)
w/ Tied Embedding	47.7	4417M (590M)
w/ Merged Attention	47.5	4049M (1180M)
w/ Cross-Attention on Global Layers Only	46.5	4233M (1180M)

Tabel 1: Hasil ablasi arsitektur untuk T5Gemma-2B-2B. Menunjukkan trade-off antara jumlah parameter dan performa. Sumber: Paper T5Gemma-2 [2]

1.1.2 Kemampuan Long Context (128K) dengan Alternating Local/Global Attention Salah satu pilar kekuatan T5Gemma-2 yang paling menonjol, terutama untuk aplikasi multi-turn chatbot, adalah kemampuannya untuk memproses konteks yang sangat panjang, yaitu hingga **128.000 token**. Kemampuan ini sangat penting dalam percakapan yang berkelanjutan, di mana riwayat dialog dapat dengan cepat menjadi sangat panjang dan melebihi batas konteks model konvensional. Untuk mengelola kompleksitas komputasional yang melekat pada mekanisme self-attention standar (yang bersifat kuadratik terhadap panjang urutan), T5Gemma-2 mengimplementasikan strategi attention yang disebut **“alternating local/global attention”**. Dalam pendekatan ini, lapisan-lapisan attention di dalam model tidak semuanya menggunakan mekanisme global attention yang menghitung hubungan antar setiap pasang token. Sebaliknya, mereka diatur secara bergantian: sekitar **5 lapisan local attention** diikuti oleh **1 lapisan global attention** [2].

Local attention membatasi jangkauan attention hanya pada jendela token yang berdekatan, yang secara dramatis mengurangi beban komputasi dan memori untuk urutan yang sangat panjang. Sementara itu, **global attention** diterapkan secara sporadis untuk memungkinkan model menangkap dependensi jarak jauh yang mungkin ada di dalam teks. Rasio 5:1 antara local dan global attention ini merupakan keseimbangan yang dirancang untuk mengoptimalkan trade-off antara efisiensi dan kemampuan pemodelan dependensi jangka panjang. Selain itu, untuk lebih meningkatkan kinerja pada konteks panjang, T5Gemma-2 menggunakan teknik yang disebut **positional interpolation**, di mana frekuensi dasar dari Rotary Position Embedding (RoPE) yang digunakan pada lapisan global attention diatur ke nilai yang sangat tinggi, yaitu **1 juta**, sementara pada lapisan local attention menggunakan nilai standar 10.000. Kombinasi dari alternating attention dan positional interpolation ini memungkinkan T5Gemma-2 untuk tidak hanya menerima input hingga 128K token selama inferensi tetapi juga untuk memberikan kinerja yang kuat pada benchmark evaluasi konteks panjang seperti RULER dan MRCR, seringkali mengungguli model decoder-only Gemma 3 yang setara [2]. Kemampuan ini memberikan fondasi yang sangat kokoh untuk membangun chatbot yang dapat mengingat dan merujuk kembali ke bagian-bagian awal dari percakapan yang sangat panjang.

1.1.3 Efektivitas Arsitektur Encoder-Decoder untuk Multi-Task Learning Arsitektur encoder-decoder yang melekat pada T5Gemma-2 memberikan serangkaian keunggulan fundamental yang membuatnya menjadi pilihan yang sangat cocok untuk skenario multi-task learning, terutama ketika tugas-tugas tersebut harus dideteksi secara implisit dari konteks. Berbeda dengan model decoder-only yang memproses seluruh input (termasuk prompt dan riwayat percakapan) secara autoregresif, model encoder-decoder memisahkan proses menjadi dua tahap yang jelas [1]. Tahap pertama adalah **encoder**, yang

memproses seluruh input secara **bidirectional**. Ini berarti encoder dapat melihat setiap token dalam konteks percakapan dari kiri ke kanan dan kanan ke kiri secara bersamaan, memungkinkannya untuk membangun representasi kontekstual yang sangat kaya dan holistik. Kemampuan ini sangat penting untuk mendeteksi intent pengguna secara implisit, karena nuansa dan maksud seringkali bergantung pada hubungan antara kata-kata di seluruh urutan, tidak hanya pada kata-kata yang mendahuluinya. Sebagai contoh, dalam percakapan multi-turn, encoder dapat menganalisis seluruh riwayat dialog untuk memahami apakah permintaan pengguna saat ini adalah permohonan untuk meringkas teks yang dibagikan beberapa putaran sebelumnya, atau permintaan untuk parafrasekan pernyataan sebelumnya.

Tahap kedua adalah **decoder**, yang bertanggung jawab untuk menghasilkan respons. Decoder beroperasi secara autoregresif, tetapi dengan satu komponen kritis: **cross-attention**. Pada setiap langkah generasi, cross-attention mekanisme memungkinkan decoder untuk “menatap kembali” (look back) pada representasi berdimensi tinggi yang dihasilkan oleh encoder. Ini memberikan decoder akses langsung ke semua informasi yang relevan dari input, memungkinkannya untuk fokus pada bagian-bagian tertentu dari percakapan saat menyusun respons. Misalnya, saat melakukan summarization, decoder dapat menggunakan cross-attention untuk mengidentifikasi dan menyoroti kalimat-kalimat kunci dalam teks asli. Dalam konteks multi-turn chatbot, ini berarti decoder dapat secara dinamis merujuk kembali ke entitas atau konsep yang disebutkan sebelumnya untuk menjawab pertanyaan atau menyelesaikan tugas NER. Efisiensi inferensi juga merupakan keuntungan besar; encoder hanya perlu memproses input panjang satu kali untuk menghasilkan representasi tetap, dan kemudian decoder dapat menggunakan representasi ini secara berulang untuk menghasilkan respons. Hal ini jauh lebih efisien daripada model decoder-only yang harus memproses ulang seluruh prefix (termasuk riwayat percakapan yang panjang) pada setiap langkah generasi token [1]. Paper T5Gemma-2 secara kuat mendukung klaim ini, menunjukkan bahwa model ini memberikan kinerja post-training yang lebih baik secara signifikan daripada model Gemma 3 decoder-only yang setara, bahkan dengan jumlah supervised fine-tuning yang minimal [2].

1.2 Keunggulan Encoder-Decoder untuk Multi-Turn Conversation

Penggunaan arsitektur encoder-decoder, yang secara spesifik diwujudkan oleh T5Gemma-2, menawarkan keunggulan metodologis yang signifikan untuk aplikasi multi-turn chatbot. Keunggulan ini berasal dari cara model memproses dan merepresentasikan informasi, yang secara alami selaras dengan dinamika percakapan. Dalam sebuah dialog, pemahaman terhadap pesan terbaru pengguna seringkali bergantung secara kritis pada seluruh riwayat percakapan yang mendahuluinya. Arsitektur encoder-decoder menangani tantangan ini dengan elegan. Seluruh riwayat percakapan, diformat dengan penanda peran seperti [USER] dan [ASSISTANT], dapat disatukan menjadi satu string input panjang dan diserahkan ke encoder. Karena encoder memproses input ini secara **bidirectional**, ia dapat membangun representasi yang mendalam yang memperhitungkan hubungan semantik dan sintaksis di antara semua bagian dialog. Ini memungkinkan model untuk memahami coreferensi (misalnya, mengetahui bahwa “itu” dalam pesan terbaru merujuk pada entitas yang diperkenalkan beberapa putaran sebelumnya), melacak topik, dan mendeteksi pergeseran intent pengguna dengan lebih efektif daripada model yang memproses input secara unidirectional.

1.2.1 Bidirectional Encoder untuk Konteks Percakapan Penuh

Inti dari keunggulan encoder-decoder terletak pada kemampuan encoder-nya untuk memproses input secara **bidirectional**. Ketika seluruh riwayat percakapan disatukan menjadi satu input string untuk encoder T5Gemma-2, setiap token dalam riwayat tersebut—mulai dari ucapan pertama sistem, pertanyaan pengguna, hingga respons asisten sebelumnya—direpresentasikan dalam konteks token-token lainnya di kedua arah. Ini berarti bahwa representasi untuk kata “ringkas” dalam permintaan pengguna akan dipengaruhi tidak hanya

oleh kata-kata yang mendahuluinya dalam kalimat yang sama tetapi juga oleh seluruh paragraf artikel yang dibagikan pengguna di turn sebelumnya. Kemampuan ini memungkinkan encoder untuk memahami nuansa percakapan yang kompleks dan mendeteksi intent task secara implisit [1]. Misalnya, jika pengguna pertama kali membagikan sebuah artikel panjang dan kemudian di turn berikutnya bertanya “Apa poin utama dari teks tadi?”, encoder dapat membuat koneksi yang kuat antara pertanyaan tersebut dan konten artikel. Ia dapat menyimpulkan bahwa tugas yang diminta adalah summarization, bahkan tanpa adanya kata kunci eksplisit seperti “tolong ringkas”. Representasi kontekstual yang kaya ini kemudian di-passing ke decoder melalui mekanisme cross-attention, memberikan dasar yang kuat untuk menghasilkan respons yang akurat dan relevan.

1.2.2 Cross-Attention Decoder untuk Fokus pada Bagian Relevan Setelah encoder membangun representasi holistik dari seluruh input, peran decoder adalah mengubah representasi tersebut menjadi respons teks yang koheren. Mekanisme kunci yang memungkinkan ini adalah **cross-attention**. Pada setiap langkah selama proses generasi, decoder tidak hanya mengandalkan informasi dari token-token yang telah dihasilkannya sebelumnya (via self-attention) tetapi juga secara aktif “bertanya” kepada output encoder untuk menentukan token berikutnya yang paling relevan. Secara spesifik, query untuk mekanisme cross-attention berasal dari representasi decoder saat ini, sementara key dan value berasal dari representasi output encoder. Ini berarti decoder dapat secara selektif fokus pada bagian-bagian tertentu dari input encoder yang paling relevan untuk tugas yang sedang dihadapi. Untuk tugas summarization, cross-attention akan cenderung memiliki bobot tinggi pada kalimat-kalimat penting dari artikel asli. Untuk tugas Named Entity Recognition (NER), cross-attention akan fokus pada bagian teks yang mengandung entitas yang disebutkan. Dalam konteks multi-turn chatbot, kemampuan ini sangat ideal. Ketika pengguna bertanya “Siapa saja entitas yang kamu sebutkan tadi?”, decoder dapat menggunakan cross-attention untuk secara dinamis menelusuri kembali riwayat percakapan dan mengekstrak nama-nama organisasi atau orang yang telah dihasilkan dalam respons sebelumnya. Fokus selektif ini memastikan bahwa output yang dihasilkan tidak hanya secara umum relevan dengan topik, tetapi juga secara spesifik didasarkan pada bukti yang ada dalam input, yang mengurangi halusinasi dan meningkatkan akurasi [1].

1.2.3 Efisiensi Inferensi pada Long Context Dalam aplikasi dunia nyata seperti chatbot, latensi dan efisiensi sumber daya adalah faktor krusial. Di sinilah arsitektur encoder-decoder T5Gemma-2 menunjukkan keunggulan komputasionalnya yang signifikan, terutama saat berhadapan dengan konteks percakapan yang panjang. Proses inferensi pada model encoder-decoder terbagi menjadi dua fase. Fase pertama, yang paling intensif secara komputasi, adalah fase **encoding**. Seluruh riwayat percakapan, yang bisa sangat panjang (hingga 128K token pada T5Gemma-2), diproses oleh encoder untuk menghasilkan satu set representasi kontekstual. Karena encoder memproses input secara paralel (non-autoregresif), fase ini, meskipun kompleks, hanya perlu dilakukan **sekali** untuk setiap respons yang akan dihasilkan. Representasi ini kemudian di-cache dan digunakan berulang kali. Fase kedua adalah fase **decoding**, di mana decoder menghasilkan respons token demi token secara autoregresif. Pada setiap langkah decoding, komputasi yang diperlukan terutama terkait dengan self-attention pada token-token output yang telah dihasilkan dan cross-attention dengan representasi encoder yang telah di-cache.

Sebaliknya, pada model **decoder-only**, seluruh input (termasuk riwayat percakapan yang panjang dan token-token yang baru dihasilkan) harus diproses ulang secara autoregresif pada **setiap langkah generasi**. Ini berarti bahwa komputasi untuk prefix yang panjang diulang lagi dan lagi, yang menyebabkan biaya komputasi meningkat secara linear dengan panjang konteks dan jumlah token yang dihasilkan. Ketika konteks percakapan menjadi sangat panjang, inefisiensi ini dapat menjadi hambatan besar. Pa-per T5Gemma-2 secara implisit mendukung keunggulan ini dengan menunjukkan kinerja yang kuat pada benchmark konteks panjang, yang menunjukkan bahwa arsitektur ini dirancang untuk menangani input

besar secara efisien [2]. Oleh karena itu, untuk chatbot multi-turn, di mana setiap turn menambah panjang konteks, arsitektur encoder-decoder T5Gemma-2 menawarkan jalur yang lebih efisien dan skalabel untuk inferensi respons yang cepat.

1.3 Konsep Instruction Tuning dan Task Prefix

Konsep instruction tuning, yang dipopulerkan melalui serangkaian paper berpengaruh seperti FLAN (Fine-tuned Language Net) dan T0, telah merevolusi cara kita melatih dan berinteraksi dengan model bahasa besar. Inti dari konsep ini adalah untuk melatih model agar tidak hanya mahir dalam tugas NLP tertentu tetapi juga mampu mengikuti instruksi yang diberikan dalam bahasa alami. Pendekatan ini mengubah model dari sistem yang terlatih untuk tugas spesifik menjadi asisten yang lebih umum dan fleksibel, yang dapat menangani berbagai macam permintaan dengan hanya membaca instruksi yang menjelaskan apa yang harus dilakukan. Konsep ini menjadi fondasi penting untuk membangun chatbot multi-task yang mampu mendeteksi dan menjalankan tugas secara implisit, karena melatih model dengan beragam instruksi membangun pemahaman yang kuat tentang hubungan antara perintah bahasa alami dan tindakan yang diperlukan.

1.3.1 T5 Text-to-Text Framework sebagai Fondasi Fondasi dari seluruh ekosistem T5, termasuk FLAN-T5 dan T5Gemma-2, adalah **Text-to-Text Transfer Transformer (T5)** framework yang diperkenalkan oleh Raffel et al. (2020). Inovasi utama dari T5 adalah kesederhanaannya yang brilian: ia merumuskan setiap tugas NLP, dari terjemahan hingga klasifikasi, sebagai masalah konversi satu teks menjadi teks lainnya. Dalam framework ini, input selalu berupa string teks, dan output yang diharapkan juga selalu berupa string teks [1]. Untuk membedakan satu tugas dari tugas lainnya, T5 memperkenalkan konsep **task prefix**. Sebuah awalan (prefix) yang spesifik untuk tugas tersebut ditambahkan di depan input sekuens. Misalnya, untuk tugas terjemahan dari bahasa Inggris ke Jerman, input akan diformat sebagai "translate English to German: The house is wonderful." dan target outputnya adalah "Das Haus ist wunderbar.". Untuk tugas peringkasan, input akan menjadi "summarize: [long article text]" dan outputnya adalah ringkasan teks tersebut. Bahkan untuk tugas klasifikasi, seperti analisis sentimen, model dilatih untuk menghasilkan label teks, misalnya input "sentiment: This movie was terrible." akan menghasilkan output "negative".

Pendekatan ini menyederhanakan arsitektur model secara drastis karena tidak lagi memerlukan kepala tugas (task-specific heads) yang berbeda untuk setiap jenis tugas. Satu model, satu fungsi kerugian (loss function), dan satu prosedur pelatihan dapat digunakan untuk semua tugas. Keuntungan praktisnya sangat besar: Anda dapat melakukan fine-tuning pada beberapa tugas secara simultan, menambahkan tugas baru tanpa perlu mengubah arsitektur model, dan memanfaatkan kemajuan dalam strategi dekode generasi teks seperti beam search atau nucleus sampling di semua aplikasi. Yang terpenting, pelatihan terpadu ini memungkinkan tugas untuk berbagi representasi. Sebuah model yang belajar menerjemahkan bahasa Inggris ke Prancis dan Jerman tidak mempelajari dua keterampilan yang tidak terkait, melainkan mempelajari representasi makna bahasa Inggris yang dapat ditransfer ke kedua arah terjemahan, yang seringkali meningkatkan kinerja pada kedua tugas tersebut [1].

1.3.2 FLAN (Fine-tuned Language Net): Pelatihan pada Ribuan Instruksi Beragam Paper "Finetuned Language Models Are Zero-Shot Learners" oleh Wei et al. (2021) memperkenalkan FLAN, yang merupakan versi T5 yang telah melalui instruction tuning. Ide utamanya adalah untuk meningkatkan kemampuan zero-shot model—kemampuannya untuk menangani tugas baru yang tidak pernah dilihat sebelumnya selama pelatihan—dengan melatihnya pada kumpulan instruksi yang sangat beragam. Wei et al. menyusun kumpulan data yang terdiri dari 62 dataset NLP yang dikelompokkan ke

dalam 12 cluster tugas, seperti Natural Language Inference, Sentiment Analysis, dan Question Answering. Mereka kemudian menggunakan berbagai template untuk mengubah contoh dari dataset-dataset ini menjadi format instruksi natural language. Misalnya, untuk tugas NLI, template instruksi bisa berupa: "Does the hypothesis follow from the premise?" atau "Premise: {premise}\nHypothesis: {hypothesis}\nDoes the hypothesis follow from the premise?".

Model T5 dilatih pada campuran dari semua tugas ini dengan instruksi-instruksi yang bervariasi. Hasilnya, FLAN menunjukkan peningkatan kinerja yang signifikan pada tugas-tugas zero-shot dibandingkan dengan T5 yang tidak melalui instruction tuning. Yang lebih menarik, paper tersebut juga menunjukkan bahwa **meningkatkan jumlah cluster tugas dalam pelatihan secara positif berkorelasi dengan peningkatan kinerja zero-shot**, yang menunjukkan bahwa model belajar untuk menggeneralisasi di seluruh berbagai jenis tugas. Paper ini secara efektif mendemonstrasikan bahwa dengan melatih model untuk mengikuti instruksi, kita dapat menciptakan model yang jauh lebih fleksibel dan mampu beradaptasi. Ini menjadi bukti awal yang kuat bahwa model dapat belajar untuk memahami “apa yang harus dilakukan” dari instruksi bahasa alami, yang merupakan langkah penting menuju sistem multi-task implicit.

1.3.3 Scaling Instruction-Finetuned Language Models (FLAN-T5) Mengikuti kesuksesan FLAN, paper “Scaling Instruction-Finetuned Language Models” oleh Chung et al. (2022) memperkenalkan FLAN-T5, yang membawa konsep instruction tuning ke skala yang jauh lebih besar. Chung et al. menyadari bahwa salah satu kunci untuk meningkatkan generalisasi dan kemampuan few-shot dari model adalah **skala dan keragaman dari tugas-tugas instruction tuning**. Mereka melatih T5 pada kumpulan data yang jauh lebih besar dan lebih beragam, yang mencakup lebih dari **1.800 tugas** yang diformat dengan lebih dari 40 juta contoh. Kumpulan tugas ini bersumber dari berbagai dataset publik seperti Super-Natural Instructions, Chain-of-Thought (CoT) datasets, dan banyak lagi. Mereka juga bereksperimen dengan berbagai ukuran model, mulai dari T5-Base hingga T5-XXL, dan berbagai tipe model, termasuk decoder-only (PaLM). Temuan utama dari paper ini adalah bahwa **instruction tuning secara konsisten meningkatkan kinerja di berbagai model dan ukuran, dan bahwa manfaatnya menjadi lebih jelas seiring dengan bertambahnya ukuran model**. FLAN-T5, hasil dari pelatihan skala besar ini, menunjukkan kinerja few-shot yang sangat kuat, bahkan seringkali mengungguli model yang jauh lebih besar (seperti PaLM 62B) pada berbagai benchmark. Paper ini juga mengidentifikasi beberapa teknik kunci yang berkontribusi pada kesuksesan FLAN-T5, termasuk penggunaan campuran tugas yang seimbang (balanced task mixing) dan penggunaan chain-of-thought data untuk meningkatkan kemampuan penalaran. Hasil dari FLAN-T5 menjadi landasan yang sangat kuat, menunjukkan bahwa pelatihan pada ribuan instruksi yang beragam adalah strategi yang sangat efektif untuk menciptakan model yang mampu menangani berbagai tugas dengan instruksi minimal.

2. Katalog Task dan Format Prefix untuk Multi-Task Implicit

Bagian ini menyajikan katalog komprehensif dari berbagai tugas NLP dan format instruksi (prefix) yang dapat digunakan untuk melatih model T5Gemma-2 Anda. Katalog ini dirancang untuk memberikan kerangka kerja yang jelas dalam membangun dataset pelatihan multi-task Anda. Fokus utama adalah pada tugas-tugas yang relevan untuk sebuah chatbot serbaguna, seperti peringkasan, parafase, pengenalan entitas, dan terjemahan. Lebih penting lagi, bagian ini juga membahas transisi dari pendekatan prefix eksplisit (seperti yang digunakan oleh T5 asli) ke instruksi natural language yang lebih fleksibel (seperti yang digunakan oleh FLAN), serta strategi untuk melatih model agar dapat beroperasi tanpa prefix sama sekali melalui deteksi intent implisit dari konteks percakapan.

2.1 Daftar Task Categories dari FLAN Collection

Repositori FLAN (Fine-tuned Language Net) Google Research menyediakan salah satu koleksi instruksi tuning publik yang paling ekstensif dan terdokumentasi dengan baik. File `flan_collection_info.csv` di dalam repositori `google-research/FLAN` secara spesifik mencantumkan **1.945 entri tugas** yang digunakan untuk melatih model seperti FLAN-T5 [4]. Analisis terhadap file ini mengungkapkan keragaman yang luar biasa dalam jenis tugas yang dapat dipelajari oleh model dalam satu kerangka kerja terpadu. Katalog tugas ini tidak hanya mencakup tugas-tugas NLP inti tetapi juga memperluas ke area seperti pemrograman, penalaran, dan dialog. Bagi pengembangan chatbot multi-task Anda, katalog ini berfungsi sebagai sumber inspirasi utama dan daftar periksa untuk memastikan bahwa model Anda terpapar pada spektrum tugas yang cukup luas selama fase pelatihan.

Kategori Tugas (Generic Task Category)	Jumlah Tugas dalam FLAN Collection	Relevansi untuk Chatbot Multi-Task
Translation	402	Sangat Tinggi: Tugas inti untuk chatbot multilingual.
Question Answering	153	Sangat Tinggi: Fungsi utama chatbot (extractive, abstractive, closed-book).
Question Generation	98	Sedang: Untuk tugas-tugas interaktif atau edukasi.
Sentiment Analysis	92	Tinggi: Untuk memahami dan merespons nuansa emosional pengguna.
Multiple-Choice QA	97	Sedang: Format umum untuk evaluasi pengetahuan dan pemahaman.
Extractive QA	42	Tinggi: Tugas QA yang sangat umum.
Text Categorization	46	Tinggi: Untuk klasifikasi topik, intensi, atau konten.
Toxic Language Detection	40	Sangat Tinggi: Kritis untuk keamanan dan moderasi konten.
Summarization	26	Sangat Tinggi: Tugas inti untuk memproses informasi panjang.
Information Extraction	34	Sangat Tinggi: Tugas inti untuk mengekstrak data terstruktur dari teks.
Named Entity Recognition	25	Sangat Tinggi: Untuk mengidentifikasi entitas kunci dalam percakapan.
Dialogue Generation	13	Sangat Tinggi: Fungsi inti untuk menjaga alur percakapan yang koheren.
Text Simplification	4	Sedang: Untuk meningkatkan aksesibilitas informasi.

Table 2 – continued

Kategori Tugas (Generic Task Category)	Jumlah Tugas dalam FLAN Collection	Relevansi untuk Chatbot Multi-Task
Paraphrasing	12	Tinggi: Untuk mengekspresikan ide dengan cara yang berbeda.
Closed-Book QA	23	Sedang: Untuk menguji pengetahuan internal model.
Coreference Resolution	18	Sedang: Untuk memahami referensi dalam percakapan multi-turn.
Grammar Error Correction	2	Rendah-Menengah: Untuk aplikasi edukasi atau penulisan.

Tabel 2: Katalog Task dari FLAN Collection yang Relevan untuk Chatbot Multi-Task. Data diolah dari `flan_collection_info.csv` [4]

2.1.1 Tugas Inti: Summarization, Paraphrase, NER, QA, Translation Dalam konteks pengembangan chatbot multi-task Anda, ada beberapa kategori tugas yang menjadi inti dari fungsionalitas yang diinginkan. Berdasarkan analisis terhadap koleksi tugas FLAN, berikut adalah rincian dan jumlah tugas untuk setiap kategori inti ini:

- 1. Summarization (Peringkasan):** Kategori ini terdiri dari **26 tugas** yang berbeda. Tugas-tugas ini bersumber dari dataset populer seperti CNN/DailyMail, XSum, SAMSum, dan Gigaword. Keberadaan 26 variasi tugas peringkasan ini menunjukkan bahwa model dapat dilatih untuk menghasilkan ringkasan dalam berbagai gaya dan panjang, mulai dari ringkasan berita yang singkat hingga abstrak akademis. Ini sangat relevan untuk chatbot Anda, di mana pengguna mungkin meminta untuk meringkas berbagai jenis teks.
- 2. Paraphrase (Parafrase):** Koleksi FLAN mencakup kategori “Paraphrasing” yang terdiri dari **12 tugas** dan “Paraphrasing Identification” dengan **3 tugas**. Tugas parafrase melibatkan penulisan ulang sebuah teks dengan kata-kata yang berbeda sambil mempertahankan makna yang sama. Ini berguna untuk chatbot untuk memperjelas poin, mengekspresikan kembali informasi dengan gaya yang berbeda, atau membantu pengguna dalam menulis. Tugas identifikasi parafrase melatih model untuk mengenali apakah dua kalimat memiliki makna yang sama, yang merupakan keterampilan pemahaman bahasa yang fundamental.
- 3. Named Entity Recognition (NER):** Kategori “Named Entity Recognition” dalam koleksi FLAN terdiri dari **25 tugas**. NER adalah tugas krusial untuk mengidentifikasi dan mengklasifikasikan entitas bernama dalam teks ke dalam kategori yang telah ditentukan sebelumnya, seperti orang, organisasi, lokasi, tanggal, dan lainnya. Dalam konteks chatbot, kemampuan NER memungkinkan model untuk mengekstrak informasi penting dari masukan pengguna. Misalnya, jika pengguna bertanya “Siapa CEO OpenAI?”, model harus dapat mengidentifikasi “OpenAI” sebagai entitas organisasi.
- 4. Question Answering (QA):** Ini adalah salah satu kategori terbesar dan paling beragam dalam koleksi FLAN. Ia mencakup **153 tugas** yang termasuk dalam kategori “Question Answering” umum, ditambah dengan sub tipe yang lebih spesifik seperti “Extractive QA” (**42 tugas**), “Closed-Book QA” (**23 tugas**), dan “Multiple-Choice QA” (**97 tugas**). Keragaman ini memungkinkan model

untuk dilatih dalam berbagai format QA, mulai dari mengekstrak jawaban langsung dari sebuah konteks hingga menjawab pertanyaan berdasarkan pengetahuan internalnya.

5. **Translation (Terjemahan):** Kategori “Translation” dalam koleksi FLAN sangat besar, terdiri dari **402 tugas**. Jumlah yang besar ini mencerminkan berbagai pasangan bahasa yang tersedia dalam dataset-dataset terjemahan. Keberadaan begitu banyak tugas terjemahan menunjukkan bahwa model yang dilatih pada koleksi ini akan memiliki pemahaman yang kuat tentang cara menerjemahkan ide dari satu bahasa ke bahasa lain. Ini relevan jika Anda berencana untuk chatbot Anda mampu berkomunikasi dalam banyak bahasa.

2.1.2 Tugas Tambahan: Dialogue, Sentiment Analysis, Text Classification Selain tugas-tugas inti, ada beberapa kategori tugas lain dalam koleksi FLAN yang, meskipun mungkin tidak secara eksplisit disebutkan dalam kebutuhan awal Anda, dapat secara signifikan meningkatkan kemampuan dan fleksibilitas chatbot Anda. Tugas-tugas tambahan ini melatih model pada aspek-aspek penting dari pemahaman dan pemrosesan bahasa yang seringkali terjadi secara implisit dalam percakapan alami.

1. **Dialogue (Dialog):** Kategori ini dalam koleksi FLAN, meskipun jumlahnya lebih kecil, mencakup tugas-tugas yang sangat relevan untuk percakapan. Contohnya termasuk “Conversational Question Answering” dan “Dialog Context Generation”. Tugas-tugas ini secara spesifik melatih model untuk memahami dinamika percakapan, di mana makna sebuah pertanyaan atau pernyataan sangat bergantung pada konteks dialog yang mendahuluinya. “Conversational Question Answering” melatih model untuk menangani serangkaian pertanyaan yang saling terkait, di mana pertanyaan selanjutnya mungkin menggunakan elipsis atau referensi balik (anaphora) yang hanya dapat dipahami dengan melihat riwayat percakapan. “Dialog Context Generation” melatih model untuk menghasilkan konteks yang sesuai untuk sebuah respons dialog, yang secara tidak langsung mengajarkannya tentang struktur dan alur percakapan yang koheren. Pelatihan pada tugas-tugas dialog ini secara langsung mendukung kemampuan multi-turn chatbot Anda.
2. **Sentiment Analysis (Analisis Sentimen):** Koleksi FLAN mencakup sejumlah besar tugas analisis sentimen, termasuk kategori “Sentiment Analysis” dengan **92 tugas** dan “Sentiment Analysis” dalam bentuk terbalik (inverted). Tugas-tugas ini melatih model untuk mengidentifikasi dan mengekstrak opini atau emosi yang terkandung dalam sebuah teks, mengklasifikasikannya sebagai positif, negatif, atau netral. Kemampuan ini sangat berharga untuk chatbot karena memungkinkannya untuk menyesuaikan nada dan isi responsnya berdasarkan sentimen yang terdeteksi dari masukan pengguna. Misalnya, jika pengguna mengungkapkan frustrasi (sentimen negatif), chatbot dapat merespons dengan lebih empati dan sopan.
3. **Text Classification (Klasifikasi Teks):** Kategori yang lebih luas dari “Text Categorization” (**46 tugas**) dan “Topic Classification” (**6 tugas**) juga sangat relevan. Tugas klasifikasi teks melibatkan penugasan sebuah teks ke satu atau lebih kategori yang telah ditentukan sebelumnya. Meskipun chatbot Anda bertujuan untuk deteksi implisit, pelatihan pada tugas klasifikasi teks membantu model untuk memahami konsep-konsep tingkat tinggi dan untuk mengelompokkan informasi. Ini dapat membantu dalam mengarahkan percakapan, mengidentifikasi topik yang dibicarakan pengguna, dan secara umum meningkatkan pemahaman kontekstual model.

2.2 Format Prefix Eksplisit (T5-style)

Format prefix eksplisit, sebagaimana diperkenalkan oleh model T5 asli, adalah pendekatan yang paling langsung untuk memberi tahu model tentang tugas yang harus dilakukannya. Pendekatan ini melibatkan penambahan string token khusus di awal setiap urutan input selama pelatihan dan inferensi. Kekuatan

dari metode ini terletak pada kesederhanaannya dan efektivitasnya yang terbukti. Dengan melatih model pada berbagai tugas yang masing-masing diawali dengan prefix yang unik, model belajar untuk mengasosiasikan setiap prefix dengan perilaku dan format output yang sesuai. Pendekatan ini masih sangat relevan dan dapat digunakan sebagai komponen dalam strategi pelatihan multi-task Anda, baik sebagai fondasi maupun sebagai bagian dari campuran data yang lebih beragam.

Kategori Tugas	Contoh Prefix Eksplisit (T5-style)	Contoh Input Lengkap	Contoh Output
Translation	translate English to German:	translate English to German: The weather is beautiful today.	Das Wetter ist heute schön.
Summarization	summarize:	summarize: The Amazon rainforest produces about 20% of the world's oxygen...	The Amazon rainforest is under threat from deforestation.
Question Answering	question: ... context: ...	question: What color is the sky? context: The sky appears blue during the day.	blue
Sentiment Analysis	sentiment:	sentiment: This movie was terrible.	negative
Named Entity Recognition	ner:	ner: Google is headquartered in Mountain View, California.	Google: ORG, Mountain View: LOC, California: LOC
Paraphrase	paraphrase:	paraphrase: The quick brown fox jumps over the lazy dog.	The fast brown fox leaps over the sleepy dog.
Text Classification	cola sentence:	cola sentence: The course is jumping well.	not acceptable

Tabel 3: Contoh Format Prefix Eksplisit untuk Berbagai Tugas. Sumber: Paper T5, Hugging Face, dan berbagai tutorial ^[1] ^[5]

2.2.1 Format untuk Summarization: `summarize:`, `tldr`; Untuk tugas peringkasan (summarization), format prefix yang paling umum dan langsung adalah **`summarize:`**. Ketika string ini ditambahkan di depan teks yang akan diringkas, model dilatih untuk menghasilkan versi yang lebih pendek dan padat dari teks tersebut. Misalnya, selama pelatihan, sebuah contoh data mungkin terlihat seperti ini:

- **Input:** summarize: The Amazon rainforest, often referred to as the lungs of the Earth, produces about 20% of the world's oxygen and is home to millions of species of plants, animals, and insects. Deforestation is threatening this vital ecosystem at an alarming rate.
- **Target Output:** Deforestation threatens the Amazon rainforest, a vital ecosystem.

Penggunaan prefix yang konsisten seperti `summarize:` memberi sinyal yang jelas kepada model bahwa tugas yang diperlukan adalah ekstraksi informasi penting dan kompresi teks. Variasi lain yang terkadang

digunakan, meskipun lebih informal, adalah **tldr;**. Akronim untuk “too long; didn’t read”, ini memberi sinyal yang sama kepada model untuk menghasilkan ringkasan. Meskipun **summarize:** lebih deskriptif dan lebih sesuai dengan gaya instruksi formal, **tldr;** dapat digunakan jika Anda ingin menambahkan variasi atau meniru gaya bahasa internet yang lebih santai dalam dataset pelatihan Anda.

2.2.2 Format untuk Translation: `translate English to German:`, `translate to French:` Untuk tugas terjemahan, format prefix harus secara spesifik menyebutkan pasangan bahasa yang terlibat. Struktur umumnya adalah **`translate [source language] to [target language]:`**. Contoh yang paling sering dikutip dalam literatur adalah **`translate English to German:`** ^[1] ^[5].

- **Input:** `translate English to German: The weather is beautiful today.`
- **Target Output:** `Das Wetter ist heute schön.`

Format ini secara eksplisit memberi tahu model tentang sumber dan target bahasa, yang memungkinkan-nya untuk mengakses pengetahuan terjemahannya yang relevan. Anda dapat mengganti bahasa-bahasa tersebut sesuai kebutuhan, misalnya `translate English to French:`, `translate Indonesian to English:`, dan sebagainya. Jika Anda berencana untuk mendukung banyak bahasa, penting untuk menyertakan berbagai variasi pasangan bahasa ini dalam dataset pelatihan Anda.

2.2.3 Format untuk NER: `ner:`, `find entities:`, `extract entities:` Meskipun model T5 asli tidak selalu memiliki tugas NER standar dengan prefix yang mapan seperti pada terjemahan atau peringkasan, kita dapat dengan mudah mendefinisikan format khusus kita sendiri untuk tujuan pelatihan multi-task. Tujuannya adalah untuk memberi sinyal kepada model bahwa ia perlu mengidentifikasi dan mengekstrak entitas bernama dari teks yang diberikan. Berikut adalah beberapa contoh format prefix yang dapat Anda gunakan:

- **`ner:`**: Ini adalah contoh yang Anda berikan dan merupakan pilihan yang sangat baik karena singkat dan jelas.
 - **Input:** `ner: Google is an American multinational technology company specializing in Internet-related services and products.`
 - **Target Output:** `Google: ORG, American: NORP, Internet: PRODUCT`
- **`find entities:`**: Ini adalah instruksi yang lebih mirip bahasa alami.
 - **Input:** `find entities: Barack Obama was the 44th president of the United States.`
 - **Target Output:** `Barack Obama: PERSON, the United States: GPE`
- **`extract entities:`**: Ini adalah variasi lain yang juga deskriptif.
 - **Input:** `extract entities: The Eiffel Tower is located in Paris, France.`
 - **Target Output:** `The Eiffel Tower: FAC, Paris: GPE, France: GPE`

Dalam contoh-contoh di atas, format output target juga perlu distandarisasi. Anda dapat menggunakan format seperti `[Entity Text]: [Entity Type]`, yang dipisahkan oleh koma, seperti yang ditunjukkan di atas.

2.2.4 Format untuk Paraphrase: `paraphrase:`, `rewrite:` Untuk tugas parafrase, tujuannya adalah untuk meminta model mengekspresikan kembali sebuah teks dengan cara yang berbeda tanpa mengubah maknanya. Mirip dengan tugas lainnya, kita dapat menggunakan prefix yang jelas dan ringkas untuk memberi sinyal tugas ini kepada model.

- **`paraphrase:`**: Ini adalah prefix yang paling umum dan intuitif untuk tugas ini.
 - **Input:** `paraphrase: The quick brown fox jumps over the lazy dog.`
 - **Target Output:** `The fast brown fox leaps over the sleepy dog.`

- **rewrite::** Ini adalah alternatif yang baik, terutama jika Anda ingin memberi kesan instruksi yang lebih umum.
 - **Input:** rewrite: She made a decision to purchase a new vehicle.
 - **Target Output:** She decided to buy a new car.

Anda juga dapat membuat variasi yang lebih spesifik untuk mengontrol gaya parafrase, misalnya **paraphrase in simpler terms:** atau **paraphrase for a child:**. Ini akan melatih model untuk tidak hanya melakukan parafrase tetapi juga menyesuaikan tingkat kesulitan bahasa sesuai dengan instruksi.

2.3 Format Instruksi Natural Language (FLAN-style)

Seiring dengan perkembangan model bahasa, terjadi pergeseran dari penggunaan prefix eksplisit yang kaku (seperti **summarize:**) ke instruksi yang lebih alami dan bervariasi (seperti **Summarize the following text:**). Pendekatan instruction tuning, yang dipopulerkan oleh FLAN dan dataset serupa, telah terbukti sangat efektif dalam meningkatkan kemampuan zero-shot dan few-shot model. Dengan melatih model pada berbagai instruksi dalam bahasa alami, model belajar untuk menginterpretasikan maksud pengguna dari berbagai cara pertanyaan atau perintah tersebut disusun. Ini adalah langkah penting menuju sistem multi-task implicit, di mana model harus mampu memahami intent dari konteks tanpa adanya trigger kata yang jelas.

Kategori Tugas	Contoh Prefix Eksplisit (T5-style)	Contoh Instruksi Natural Language (FLAN-style)
Summarization	summarize: [text]	Summarize the following text: [text] Provide a brief summary of the article below: [text] What is the main point of this passage? [text]
Translation	translate English to German: [text]	Translate the following sentence into German: [text] How would you say this in French? [text] Provide the Spanish translation for: [text]
Question Answering	question: [q] context: [c]	Answer the question based on the context provided: [c] Question: [q] Given the following passage, what is the answer to: [q]? [c] Based on the text, [q]? [c]
Named Entity Recognition	ner: [text]	Extract all named entities from the following text: [text] Identify the people, organizations, and locations mentioned in: [text] Find all proper nouns in the sentence below and label their type: [text]

Table 4 – continued

Kategori Tugas	Contoh Prefix Eksplisit (T5-style)	Contoh Instruksi Natural Language (FLAN-style)
Paraphrase	paraphrase: [text]	Rewrite the following sentence in your own words: [text] Express the following idea differently: [text] Simplify the following sentence: [text]

Tabel 4: Perbandingan Format Prefix Eksplisit dan Instruksi Natural Language. Sumber: Berbagai paper FLAN dan tutorial ^[1] ^[6]

2.3.1 Contoh Instruksi Summarization: Summarize the following text:, Provide a brief summary:

Alih-alih hanya menggunakan `summarize:`, Anda dapat melatih model Anda dengan berbagai instruksi peringkasan yang lebih alami. Ini membantu model untuk memahami konsep peringkasan bahkan ketika diminta dengan cara yang berbeda.

- Summarize the following text: [text]
- Provide a brief summary of the article below: [text]
- What is the main point of this passage? [text]
- Can you give me a short overview of this document? [text]
- In a few sentences, describe what the following text is about: [text]

Dengan menyertakan variasi instruksi ini dalam dataset pelatihan, model Anda akan menjadi lebih kuat dan mampu menangani permintaan peringkasan yang tidak mengikuti format `summarize:` yang kaku.

2.3.2 Contoh Instruksi NER: Identify all named entities:, Find the names of people and organizations: Untuk NER, instruksi natural language dapat membantu model memahami tugas ini bahkan ketika tidak ada prefix `ner:` yang eksplisit. Ini sangat penting untuk deteksi implisit.

- Identify all named entities in the following text: [text]
- Find the names of people, organizations, and locations mentioned in: [text]
- Extract all proper nouns from the sentence below and label their type: [text]
- Who and what is mentioned in the following passage? [text]
- List all the important entities in this text: [text]

Instruksi-instruksi ini memberikan konteks yang lebih kaya tentang apa yang diharapkan dari model, yang dapat meningkatkan kinerja, terutama dalam skenario zero-shot di mana model belum pernah melihat instruksi yang tepat sebelumnya tetapi memahami konsep umum dari “entitas” dan “mengidentifikasi”.

2.3.3 Contoh Instruksi Paraphrase: Rewrite the following sentence:, Express this in a different way: Untuk parafrase, instruksi natural language dapat membantu model menangkap nuansa dari permintaan tersebut, yang mungkin tidak tercakup oleh prefix `paraphrase:` yang sederhana.

- Rewrite the following sentence in your own words: [text]
- Express this idea in a different way: [text]
- Simplify the following sentence so that a child can understand it: [text]
- Rephrase the following statement: [text]

- Say the same thing but using different words: [text]

Dengan menggunakan variasi instruksi ini, Anda tidak hanya melatih model untuk melakukan parafrase tetapi juga untuk memahami berbagai nuansa permintaan tersebut, seperti penyederhanaan atau perubahan gaya, yang dapat terjadi secara implisit dalam percakapan.

2.4 Strategi Transisi ke Implicit Task Detection (Tanpa Prefix)

Tujuan akhir Anda adalah membuat chatbot yang dapat menentukan tugas apa yang harus dilakukannya tanpa instruksi eksplisit. Ini adalah tingkat kecerdasan tertinggi dalam sistem multi-task, di mana model menggunakan konteks percakapan untuk secara dinamis menyimpulkan intent pengguna. Transisi dari sistem berbasis prefix ke sistem implicit memerlukan strategi pelatihan yang cermat. Strategi ini harus secara bertahap mengajarkan model untuk mengandalkan sinyal kontekstual daripada sinyal eksplisit untuk mengidentifikasi tugas. Berikut adalah beberapa strategi yang dapat Anda terapkan, yang didukung oleh prinsip-prinsip dari paper-paper instruction tuning dan multi-task learning.

2.4.1 Pelatihan Campuran: Data dengan dan tanpa Prefix Eksplisit Strategi pertama dan paling fundamental adalah **mixed training** atau pelatihan campuran. Strategi ini melibatkan pembuatan dataset pelatihan yang merupakan campuran dari tiga jenis contoh:

1. **Contoh dengan Prefix Eksplisit:** Sebagian besar data pelatihan Anda (misalnya, 50-60%) masih harus menggunakan format prefix eksplisit seperti `summarize:`, `ner:`, atau instruksi natural language yang jelas seperti `Summarize the following text:`. Ini memastikan bahwa model memiliki pemahaman yang kuat dan jelas tentang tugas-tugas dasar.
2. **Contoh dengan Instruksi Implisit:** Sebagian data pelatihan Anda (misalnya, 20-30%) harus diformat tanpa prefix tugas apa pun. Sebaliknya, tugas harus disiratkan oleh konteks. Ini adalah bagian kunci dari pelatihan.
 - **Contoh Summarization Implisit:**
 - **Input:** [USER] Artificial Intelligence telah berkembang pesat dalam dekade terakhir... [ASSISTANT]
 - **Target Output:** AI telah berkembang pesat berkat deep learning dan investasi besar dari perusahaan teknologi.
 - **Contoh NER Implisit:**
 - **Input:** [USER] Siapa saja entitas yang kamu sebutkan tadi? [ASSISTANT] (Asumsikan sebelumnya ada konteks tentang Google, OpenAI, Meta).
 - **Target Output:** Entities: Google (Organization), OpenAI (Organization), Meta (Organization)
3. **Contoh Percakapan Multi-Turn:** Sebagian kecil data pelatihan Anda (misalnya, 10-20%) harus berupa contoh percakapan multi-turn yang lengkap, di mana tugas berganti dari satu turn ke turn berikutnya tanpa adanya perubahan prefix yang eksplisit. Ini adalah bentuk pelatihan paling canggih untuk deteksi implisit.
 - **Turn 1 (Chat):** User: "Halo, apa kabar?" -> Assistant: "Baik, terima kasih. Ada yang bisa saya bantu?"
 - **Turn 2 (Summarization):** User: "Bisa bantu saya memahami artikel ini? [paste artikel]" -> Assistant: [Output ringkasan]
 - **Turn 3 (Paraphrase):** User: "Bagus! Bisa buat versi yang lebih sederhana?" -> Assistant: [Output parafrase]

- **Turn 4 (NER):** User: "Siapa saja entitas yang kamu sebutkan tadi?" -> Assistant:
[Output NER]

Dengan mencampur ketiga jenis contoh ini, Anda secara bertahap mengajarkan model untuk tidak hanya mengandalkan prefix tugas tetapi juga untuk menggunakan konteks percakapan untuk menyimpulkan tugas yang dimaksud.

2.4.2 Penggunaan Conversation History sebagai Signal Intent Dalam sebuah chatbot multi-turn, **conversation history** adalah sinyal yang paling kuat untuk deteksi intent implisit. Model harus dilatih untuk melihat riwayat percakapan sebagai bagian integral dari inputnya. Untuk mencapai ini, Anda harus memformat data pelatihan Anda dengan jelas untuk menyertakan riwayat.

Format yang umum digunakan adalah dengan menambahkan penanda peran (role markers) seperti [USER] dan [ASSISTANT] di depan setiap bagian dari dialog.

```
Input untuk Encoder:
[SYSTEM] Anda adalah asisten AI yang serbaguna.
[USER] Halo, saya baru saja menulis artikel tentang AI. Bisa bantu review?
[ASSISTANT] Tentu! Silakan share artikelnya, saya akan bantu review.
[USER] Artificial Intelligence telah berkembang pesat dalam dekade terakhir...
```

Dengan melatih model pada input yang berisi seluruh riwayat percakapan, encoder bidirectional dapat mempelajari hubungan antara turn-turn yang berbeda. Model akan belajar bahwa jika pengguna membagikan teks panjang di satu turn, dan kemudian di turn berikutnya bertanya “Apa poin utamanya?”, maka tugas yang dimaksud adalah summarization, bahkan tanpa adanya kata “ringkas”. Sistem prompt, seperti Anda adalah asisten AI yang serbaguna., juga dapat memberikan sinyal tambahan dengan menjelaskan kemampuan model, yang membantu model dalam mengaitkan permintaan pengguna dengan salah satu kemampuannya.

2.4.3 System Prompt untuk Menjelaskan Kemampuan Model **System prompt** adalah instruksi tingkat tinggi yang diberikan kepada model di awal percakapan untuk mengatur perilaku dan konteksnya. Ini adalah alat yang sangat ampuh untuk membantu deteksi intent implisit. System prompt Anda harus secara jelas dan ringkas menjelaskan semua kemampuan atau tugas yang dapat dilakukan oleh chatbot Anda.

Contoh system prompt yang efektif: Anda adalah asisten AI yang serbaguna yang dapat meringkas teks, memparafrase kalimat, mengidentifikasi entitas bernama, menjawab pertanyaan, menerjemahkan bahasa, dan melakukan percakapan normal.

Dengan menyertakan system prompt ini di awal setiap contoh pelatihan Anda, Anda memberikan konteks awal yang kuat. Ini membantu model untuk “mengingat” kemampuannya saat ia memproses riwayat percakapan. Ketika pengguna kemudian membuat permintaan yang ambigu atau implisit, model dapat mencocokkannya dengan daftar kemampuan yang diberikan oleh system prompt. Ini seperti memberikan daftar menu kepada model, sehingga ketika pelanggan (pengguna) memesan sesuatu dengan eufemisme, pelayan (model) masih dapat memahami apa yang dimaksud dari daftar menu tersebut. Strategi ini, yang dikombinasikan dengan pelatihan campuran dan pemanfaatan riwayat percakapan, menciptakan pendekatan multi-pronged yang kuat untuk melatih chatbot multi-task Anda agar dapat beroperasi secara implicit.

3. Strategi Fine-Tuning Berdasarkan Riset Akademik

Mengimplementasikan ide Anda tentang chatbot multi-task implicit memerlukan strategi fine-tuning yang cermat dan berbasis bukti. Riset akademik telah menghasilkan berbagai teknik dan metodologi yang dapat secara signifikan meningkatkan efisiensi dan efektivitas proses fine-tuning. Bagian ini merangkum strategi-strategi kunci yang didukung oleh literatur ilmiah, mulai dari efisiensi komputasi dengan Parameter-Efficient Fine-Tuning (PEFT), strategi manajemen data yang optimal, hingga metodologi pelatihan multi-turn yang spesifik untuk tugas Anda.

3.1 Parameter-Efficient Fine-Tuning (PEFT) dengan LoRA

Salah satu tantangan terbesar dalam fine-tuning model bahasa besar adalah biaya komputasional dan kebutuhan sumber daya yang sangat tinggi. Untuk mengatasi ini, komunitas riset telah mengembangkan berbagai teknik Parameter-Efficient Fine-Tuning (PEFT). Tujuannya adalah untuk mencapai kinerja yang sebanding dengan fine-tuning penuh (full fine-tuning) tetapi dengan memperbarui hanya sebagian kecil dari parameter model. Di antara teknik-teknik PEFT, **Low-Rank Adaptation (LoRA)** telah menjadi salah satu yang paling populer dan terbukti efektif, terutama untuk arsitektur Transformer.

3.1.1 Konfigurasi LoRA untuk Arsitektur Encoder-Decoder Konsep dasar LoRA, yang diperkenalkan oleh Hu et al. (2021), adalah bahwa perubahan pada bobot model selama fine-tuning memiliki “rank” yang intrinsik rendah. Alih-alih memperbarui seluruh matriks bobot W yang besar, LoRA memperkenalkan dua matriks dekomposisi peringkat rendah (low-rank decomposition), A dan B , yang jauh lebih kecil. Selama fine-tuning, bobot asli W dibekukan (frozen), dan hanya matriks A dan B yang dilatih. Output dari lapisan yang dimodifikasi kemudian menjadi $h = Wx + BAx$, di mana x adalah input dan h adalah output. Pada saat inferensi, bobot yang telah disesuaikan dapat dihitung sebagai $W' = W + BA$, yang memungkinkan penggabungan perubahan tanpa menambahkan latensi inferensi.

Untuk arsitektur encoder-decoder seperti T5Gemma-2, LoRA dapat diterapkan pada berbagai bagian model. Yang paling umum adalah menerapkannya pada matriks proyeksi di dalam mekanisme attention, yaitu matriks query (`q_proj`), key (`k_proj`), value (`v_proj`), dan output (`o_proj`). Anda juga dapat menerapkannya pada lapisan feed-forward (`gate_proj`, `up_proj`, `down_proj`). Pilihan target modul ini adalah hiperparameter penting. Sebuah konfigurasi yang umum dan sering berhasil adalah:

- `r` (rank): **16** atau **32** (rank yang lebih tinggi meningkatkan kapasitas ekspresif tetapi juga jumlah parameter yang dilatih).
- `lora_alpha`: **32** atau **64** (parameter skala yang mengontrol magnitude dari adaptasi).
- `lora_dropout`: **0.05** (untuk regularisasi).
- `target_modules`: [`"q_proj"`, `"k_proj"`, `"v_proj"`, `"o_proj"`] (menerapkan LoRA pada semua matriks proyeksi attention).
- `task_type`: **SEQ_2_SEQ_LM** (ini adalah pengidentifikasi penting yang memberi tahu PEFT library bahwa Anda sedang bekerja dengan model sequence-to-sequence).

3.1.2 Target Moduls untuk Merged Attention di T5Gemma-2 T5Gemma-2 memperkenalkan konsep **Merged Attention**, yang menggabungkan self-attention dan cross-attention menjadi satu modul attention tunggal [2]. Ini adalah perbedaan arsitektural penting dari T5 standar dan dapat memengaruhi cara Anda menerapkan LoRA. Dalam arsitektur standar, self-attention dan cross-attention memiliki set parameter sendiri. Namun, dalam Merged Attention, mereka berbagi parameter W_q , W_k , W_v , dan W_o . Ini

berarti bahwa saat Anda menerapkan LoRA pada `q_proj`, `k_proj`, `v_proj`, dan `o_proj`, adaptasi tersebut secara otomatis akan memengaruhi kedua jenis attention.

Dalam praktiknya, ini menyederhanakan konfigurasi LoRA karena Anda tidak perlu membedakan antara modul self-attention dan cross-attention. Anda tetap menargetkan modul-modul proyeksi yang sama (`q_proj`, `k_proj`, `v_proj`, `o_proj`), dan PEFT library (seperti Hugging Face PEFT) akan secara otomatis menerapkan adaptasi LoRA pada parameter yang bersama-sama digunakan oleh mekanisme Merged Attention. Ini memastikan bahwa adaptasi yang dipelajari akan memengaruhi cara model memproses input (melalui self-attention) dan cara ia menghubungkan input dengan output yang dihasilkan (melalui cross-attention), yang sangat penting untuk tugas-tugas multi-turn di mana pemahaman konteks dan generasi respons yang relevan saling terkait erat.

3.1.3 Keunggulan LoRA dalam Mencegah Catastrophic Forgetting Salah satu risiko utama dalam fine-tuning model yang telah terprestasi adalah **catastrophic forgetting**, yaitu kecenderungan model untuk kehilangan pengetahuan dan kemampuan yang telah dipelajarinya selama pre-training saat ia dilatih pada tugas baru yang spesifik. Ini sangat relevan dalam skenario multi-task Anda, di mana Anda ingin model tidak hanya mempelajari tugas-tugas baru (sebagai chatbot) tetapi juga mempertahankan kemampuannya yang luas yang diperoleh selama pre-training. LoRA secara inheren membantu dalam mengurangi masalah ini. Karena LoRA hanya memperbarui sebagian kecil parameter (biasanya kurang dari 1% dari total parameter model), sebagian besar bobot asli yang telah dipelajari selama pre-training tetap utuh dan tidak berubah [1].

Dengan membekukan bobot asli dan hanya melatih matriks adaptasi peringkat rendah, Anda secara efektif memberi tahu model: “Pertahankan semua yang telah kamu pelajari, dan tambahkan pengetahuan baru ini di atasnya.” Ini memastikan bahwa model tetap memiliki pemahaman umum yang kuat tentang bahasa, pengetahuan dunia, dan kemampuan penalaran yang diperolehnya dari dataset pre-training yang masif. Selain itu, dalam strategi pelatihan multi-task Anda, Anda dapat dengan mudah mencampur data dari tugas-tugas umum (misalnya, data dari FLAN Collection) dengan data percakapan multi-turn spesifik Anda. Penggunaan LoRA pada campuran data ini akan memungkinkan model untuk menyesuaikan diri dengan domain chatbot sambil mempertahankan kemampuannya yang lebih luas, karena bobot dasar yang mendukung kemampuan-kemampuan tersebut tetap stabil.

3.2 Konstruksi Dataset Multi-Task Multi-Turn

Keberhasilan fine-tuning model Anda sangat bergantung pada kualitas, keragaman, dan format dari dataset pelatihan. Untuk chatbot multi-task implicit, dataset Anda tidak hanya harus mencakup berbagai tugas tetapi juga harus memodelkan dinamika percakapan alami, di mana tugas-tugas tersebut muncul secara implisit dari konteks. Bagian ini membahas strategi kunci untuk membangun dataset semacam itu, berdasarkan prinsip-prinsip dari riset multi-task learning dan pemodelan dialog.

3.2.1 Konversi Dataset Task-Specific ke Format Percakapan Natural Langkah pertama adalah mengubah dataset tugas-spesifik yang ada (seperti SQuAD untuk QA, CoNLL-2003 untuk NER, atau CNN/DailyMail untuk summarization) menjadi format percakapan multi-turn yang terlihat alami. Proses ini melibatkan penambahan konteks percakapan di sekitar contoh tugas inti. Sebagai contoh, sebuah pasangan pertanyaan-jawaban dari SQuAD dapat diubah menjadi serangkaian turn percakapan:

- **Data Asli (SQuAD):**
 - Konteks: “The University of Notre Dame du Lac is a private Catholic research university in Indiana.”

- Pertanyaan: “What type of university is the University of Notre Dame?”
- Jawaban: “private Catholic research university”
- **Data Konversi (Format Percakapan):**
 - System: “Anda adalah asisten yang dapat menjawab pertanyaan berdasarkan teks.”
 - User: “Bisakah Anda membantu saya memahami teks ini? The University of Notre Dame du Lac is a private Catholic research university in Indiana.”
 - Assistant: “Tentu, saya siap membantu. Apa yang ingin Anda ketahui tentang teks tersebut?”
 - User: “Jenis universitas apa yang dimiliki Notre Dame?”
 - Assistant: “private Catholic research university”

Proses konversi ini harus dilakukan untuk setiap jenis tugas yang ingin Anda dukung. Untuk tugas summarization, Anda dapat membuat dialog di mana pengguna meminta bantuan untuk memahami sebuah artikel panjang. Untuk NER, dialog dapat dimulai dengan pengguna yang meminta untuk mengidentifikasi nama-nama penting dalam sebuah teks. Keragaman dalam cara dialog dimulai dan tugas diminta akan sangat membantu model dalam mempelajari deteksi intent implisit.

3.2.2 Strategi Mixing: Examples-Proportional dan Temperature-Scaled Setelah Anda memiliki dataset untuk setiap tugas individual, Anda perlu menggabungkannya menjadi satu dataset pelatihan yang besar. Strategi pencampuran (mixing strategy) sangat penting untuk memastikan bahwa model tidak menjadi bias terhadap tugas yang memiliki lebih banyak data. Paper T5 asli dan FLAN menyarankan beberapa strategi. Dua strategi yang paling umum adalah:

1. **Examples-Proportional Mixing:** Strategi ini sederhana: Anda mencampur contoh dari setiap tugas secara proporsional dengan ukuran dataset aslinya. Misalnya, jika tugas A memiliki 1 juta contoh dan tugas B memiliki 100.000 contoh, maka dalam setiap batch pelatihan, Anda akan memiliki 10 kali lebih banyak contoh dari tugas A daripada tugas B. Kelemahan dari strategi ini adalah dapat mendominasi tugas-tugas dengan dataset yang sangat besar, yang mungkin bukan tugas yang paling penting untuk chatbot Anda.
2. **Temperature-Scaled Mixing:** Strategi ini memberikan lebih banyak kontrol. Anda dapat menetapkan probabilitas sampling untuk setiap tugas secara manual. Ini memungkinkan Anda untuk meningkatkan frekuensi tugas-tugas penting (seperti chat normal, summarization, dan QA) meskipun dataset aslinya mungkin lebih kecil. Sebagai contoh, Anda dapat menetapkan probabilitas sebagai berikut:

- summarization: 0.20
- paraphrase: 0.15
- ner: 0.15
- qa: 0.20
- translation: 0.10
- general_chat: 0.20 (ini sangat penting untuk menjaga kemampuan percakapan)

Dengan menggunakan strategi ini, Anda memastikan bahwa model mendapat eksposur yang cukup ke semua tugas penting, terlepas dari ukuran dataset asli mereka.

3.2.3 Loss Masking: Fokus pada Assistant Responses Dalam pelatihan model bahasa, fungsi kerugian (loss function) digunakan untuk mengukur seberapa jauh output model dari target yang diinginkan. Dalam konteks chatbot, Anda ingin model belajar untuk menghasilkan respons asisten yang baik, bukan untuk memprediksi apa yang akan dikatakan pengguna berikutnya. Oleh karena itu, strategi **loss masking** yang tepat sangat penting.

Strategi yang paling umum dan direkomendasikan, seperti yang dibahas dalam paper dari Together AI, adalah **full instruction masking**. Ini berarti Anda hanya menghitung loss pada token-token yang merupakan bagian dari respons asisten. Semua token lainnya dalam urutan, termasuk system prompt dan semua turn pengguna, di-mask (diabaikan) dan tidak berkontribusi pada perhitungan loss. Dalam praktiknya, ini dapat diimplementasikan dengan mengatur label untuk token-token yang ingin Anda mask ke nilai khusus seperti `-100`, yang diabaikan oleh fungsi kerugian standar seperti `CrossEntropyLoss`.

Sebagai contoh, untuk urutan berikut: [SYSTEM] Anda adalah asisten. [USER] Halo! [ASSISTANT] Hai, ada yang bisa saya bantu?

Anda hanya akan menghitung loss pada token-token: Hai, ada yang bisa saya bantu?. Token-token lainnya ([SYSTEM] Anda adalah asisten. [USER] Halo!) akan memiliki label `-100`. Strategi ini memastikan bahwa seluruh fokus pembelajaran model adalah pada generasi respons yang relevan dan koheren, yang merupakan inti dari fungsionalitas chatbot.

3.3 Konfigurasi Training

Bagian terakhir dari strategi fine-tuning Anda adalah menetapkan konfigurasi pelatihan yang optimal. Ini mencakup pemilihan hiperparameter seperti learning rate, ukuran batch, dan durasi pelatihan. Konfigurasi yang tepat akan memastikan bahwa model belajar secara efektif dari data Anda tanpa overfitting atau underfitting.

3.3.1 Rekomendasi Hyperparameter: Learning Rate, Batch Size Pemilihan hiperparameter adalah lebih sebuah seni daripada ilmu, dan seringkali memerlukan beberapa eksperimen. Namun, ada beberapa pedoman umum yang didukung oleh praktik terbaik dalam komunitas.

- **Learning Rate:** Untuk fine-tuning dengan LoRA, learning rate yang lebih tinggi daripada yang digunakan untuk full fine-tuning seringkali efektif. Sebuah nilai awal yang umum adalah **`2e-4`**. Anda juga dapat menggunakan strategi learning rate schedule seperti “cosine decay with warmup”, di mana learning rate dimulai dari nol, dinaikkan secara linear selama fase warmup (misalnya, 10% dari total langkah pelatihan), dan kemudian diturunkan mengikuti kurva kosinus. Ini membantu model untuk menstabilkan pembelajaran di awal dan kemudian menyempurnakan bobotnya secara bertahap.
- **Batch Size:** Ukuran batch yang efektif (effective batch size) adalah hasil kali dari ukuran batch per perangkat dan jumlah langkah akumulasi gradien. Karena keterbatasan memori GPU, Anda mungkin tidak dapat menggunakan ukuran batch yang besar secara langsung. Sebagai gantinya, Anda dapat menggunakan teknik **gradient accumulation**. Misalnya, jika Anda menetapkan `per_device_train_batch_size=2` dan `gradient_accumulation_steps=8`, maka gradien akan diakumulasi selama 8 langkah sebelum satu pembaruan bobot dilakukan. Ini menciptakan ukuran batch efektif sebesar $2 * 8 = 16$, yang seringkali memberikan stabilitas pelatihan yang lebih baik.
- **Durasi Pelatihan:** Jumlah epoch pelatihan (misalnya, **3 epoch**) adalah titik awal yang umum. Namun, lebih penting untuk memantau metrik evaluasi Anda (seperti validation loss) dan menggunakan teknik seperti **early stopping** untuk menghentikan pelatihan jika kinerja pada set validasi mulai menurun, yang merupakan tanda overfitting.

3.3.2 Penggunaan BF16 dan Gradient Checkpointing untuk Efisiensi Fine-tuning model bahasa besar, bahkan dengan LoRA, masih membutuhkan sumber daya komputasi yang signifikan. Ada beberapa teknik yang dapat Anda gunakan untuk mengoptimalkan efisiensi pelatihan dan mengurangi penggunaan memori.

- **Mixed Precision Training (BF16):** T5Gemma-2 secara native mendukung format floating point **bfloat16 (BF16)**. Menggunakan BF16 alih-alih float32 (FP32) dapat secara dramatis mengurangi penggunaan memori dan mempercepat pelatihan, terutama pada GPU modern yang memiliki dukungan hardware untuk BF16 (seperti GPU NVIDIA A100 atau H100). Dalam konfigurasi pelatihan Anda, Anda dapat mengaktifkannya dengan menetapkan `bf16=True`.
- **Gradient Checkpointing:** Ini adalah teknik yang memperdagangkan komputasi untuk memori. Alih-alih menyimpan semua aktivasi perantara selama forward pass (yang diperlukan untuk perhitungan gradien selama backward pass), gradient checkpointing hanya menyimpan sebagian kecil dari aktivasi tersebut. Aktivasi lainnya akan dihitung ulang sesuai kebutuhan selama backward pass. Ini dapat secara signifikan mengurangi penggunaan memori, memungkinkan Anda untuk menggunakan ukuran batch yang lebih besar atau model yang lebih besar dengan GPU yang sama. Anda dapat mengaktifkannya dengan `gradient_checkpointing=True`.

Dengan menggabungkan strategi PEFT (LoRA), format data multi-turn, dan konfigurasi pelatihan yang dioptimalkan, Anda dapat secara efektif fine-tune model T5Gemma-2 Anda untuk menjadi chatbot multi-task yang canggih dan mampu beroperasi secara implicit.

4. Evaluasi dan Metrik untuk Multi-Task Chatbot

Setelah melatih model Anda, langkah selanjutnya yang sama pentingnya adalah evaluasi. Anda perlu mengukur seberapa baik model Anda menangani masing-masing tugas individual dan juga seberapa baik ia beroperasi sebagai chatbot multi-turn yang koheren. Evaluasi ini harus dilakukan secara sistematis menggunakan metrik yang relevan untuk setiap jenis tugas. Kombinasi dari metrik otomatis untuk tugas individu dan evaluasi manusia untuk kualitas percakapan secara keseluruhan akan memberikan gambaran yang paling komprehensif tentang kinerja model Anda.

4.1 Metrik Evaluasi per Tugas

Untuk tugas-tugas yang lebih terstruktur, di mana ada jawaban yang benar atau target yang jelas, Anda dapat menggunakan berbagai metrik evaluasi otomatis. Metrik-metrik ini memungkinkan Anda untuk secara kuantitatif membandingkan output model Anda dengan jawaban referensi (ground truth).

4.1.1 Summarization: ROUGE-1, ROUGE-2, ROUGE-L **ROUGE (Recall-Oriented Understudy for Gisting Evaluation)** adalah serangkaian metrik yang paling umum digunakan untuk mengevaluasi kualitas teks yang dihasilkan oleh model, terutama untuk tugas peringkasan. ROUGE bekerja dengan membandingkan n-gram (urutan kata) dalam output model dengan n-gram dalam satu atau lebih ringkasan referensi yang dibuat oleh manusia.

- **ROUGE-1:** Mengukur tumpang tindih (overlap) dari **unigram** (kata individu) antara output sistem dan ringkasan referensi. Ini memberikan indikasi umum tentang seberapa banyak kata kunci yang berhasil diambil oleh model.
- **ROUGE-2:** Mengukur tumpang tindih dari **bigram** (pasangan kata berurutan). Ini adalah metrik yang lebih ketat daripada ROUGE-1 karena memperhitungkan urutan kata. Skor ROUGE-2 yang tinggi menunjukkan bahwa model tidak hanya menggunakan kata-kata yang tepat tetapi juga menyusunnya dengan cara yang mirip dengan ringkasan referensi.
- **ROUGE-L:** Mengukur tumpang tindih urutan terpanjang (longest common subsequence/LCS) antara output sistem dan referensi. LCS adalah urutan kata terpanjang yang muncul dalam kedua teks dalam urutan yang sama, tetapi tidak harus berurutan. ROUGE-L memberikan gambaran

yang lebih fleksibel tentang kesamaan struktural karena tidak memerlukan kecocokan n-gram yang ketat.

Skor ROUGE biasanya dilaporkan sebagai F1-score, yang merupakan rata-rata harmonis dari precision dan recall. Skor yang lebih tinggi menunjukkan kualitas ringkasan yang lebih baik.

4.1.2 Paraphrase: BLEU, BERTScore Evaluasi parafrase sedikit lebih rumit karena ada banyak cara yang valid untuk mengekspresikan ide yang sama. Oleh karena itu, metrik yang lebih canggih seringkali lebih disukai.

- **BLEU (Bilingual Evaluation Understudy):** Meskipun awalnya dikembangkan untuk terjemahan mesin, BLEU juga dapat digunakan untuk parafrase. Ini mengukur kualitas teks yang dihasilkan dengan membandingkan n-gram dalam output dengan satu atau lebih referensi. Namun, BLEU cenderung terlalu menghukum variasi kreatif yang valid, sehingga mungkin bukan metrik utama yang terbaik untuk parafrase.
- **BERTScore:** Ini adalah metrik evaluasi yang lebih baru dan lebih canggih. Alih-alih mengandalkan kecocokan n-gram yang tepat, BERTScore menggunakan embeddings kontekstual dari model BERT untuk menghitung kesamaan semantik antara token-token dalam output sistem dan token-token dalam referensi. Ini memungkinkannya untuk menangkap kesamaan makna bahkan ketika kata-kata yang berbeda digunakan, yang membuatnya menjadi metrik yang lebih baik untuk mengevaluasi parafrase. BERTScore melaporkan precision, recall, dan F1-score berdasarkan kesamaan embedding.

4.1.3 NER: F1-score (Entity-Level) Evaluasi untuk Named Entity Recognition (NER) adalah masalah klasifikasi urutan. Tujuannya adalah untuk mengidentifikasi dan mengklasifikasikan setiap entitas dengan benar. Metrik evaluasi standar adalah **F1-score**, yang dihitung pada level entitas.

Untuk setiap entitas yang diidentifikasi oleh model, dibandingkan dengan entitas dalam anotasi referensi (ground truth). Ada tiga kemungkinan hasil:

- **True Positive (TP):** Model mengidentifikasi entitas yang benar dan memberinya label yang benar.
- **False Positive (FP):** Model mengidentifikasi sesuatu sebagai entitas yang sebenarnya bukan entitas, atau memberinya label yang salah.
- **False Negative (FN):** Model gagal mengidentifikasi entitas yang ada dalam referensi.

Berdasarkan ini, kita dapat menghitung:

- **Precision:** $TP / (TP + FP)$ (dari semua entitas yang diidentifikasi model, berapa banyak yang benar?).
- **Recall:** $TP / (TP + FN)$ (dari semua entitas yang seharusnya diidentifikasi, berapa banyak yang berhasil diidentifikasi oleh model?).
- **F1-score:** $2 * (Precision * Recall) / (Precision + Recall)$ (rata-rata harmonis dari precision dan recall).

F1-score adalah metrik tunggal yang memberikan keseimbangan antara precision dan recall, dan merupakan metrik standar untuk mengevaluasi kinerja sistem NER.

4.1.4 QA: Exact Match, F1 Evaluasi untuk Question Answering (QA) dapat bervariasi tergantung pada apakah tugasnya adalah extractive (mengestrak jawaban dari konteks) atau abstractive (menghasilkan jawaban dari pengetahuan internal model). Untuk extractive QA, dua metrik utama yang

digunakan adalah:

- **Exact Match (EM):** Ini adalah metrik yang sangat ketat. Ini mengukur persentase prediksi model yang cocok persis dengan salah satu jawaban referensi. Ini hanya memberikan skor 1 jika string prediksi identik dengan string referensi, dan 0 jika tidak. Ini adalah metrik yang bagus untuk tugas QA di mana jawaban yang benar adalah entitas atau frasa yang spesifik.
- **F1 Score:** Metrik ini lebih memaafkan daripada Exact Match. Ia memperlakukan prediksi dan jawaban referensi sebagai sekumpulan token dan menghitung F1-score (rata-rata harmonis dari precision dan recall) berdasarkan tumpang tindih token. Ini memungkinkan untuk variasi kecil dalam jawaban yang masih secara semantik benar (misalnya, “the United States” vs. “United States”). F1-score seringkali memberikan gambaran yang lebih baik tentang kinerja model secara keseluruhan.

4.2 Evaluasi Kualitas Percakapan Multi-Turn

Meskipun metrik otomatis penting untuk tugas individual, mereka tidak dapat sepenuhnya menangkap kualitas keseluruhan dari pengalaman berbicara dengan chatbot. Sebuah chatbot yang sangat baik tidak hanya melakukan tugasnya dengan benar tetapi juga melakukannya dengan cara yang koheren, kontekstual, dan menyenangkan. Oleh karena itu, evaluasi manusia tetap menjadi standar emas untuk menilai kualitas percakapan multi-turn.

4.2.1 Evaluasi Manusia: Coherence, Relevance, Fluency Evaluasi manusia melibatkan penilaian subjektif terhadap respons chatbot oleh manusia. Beberapa dimensi kunci yang dievaluasi meliputi:

- **Coherence (Koherensi):** Seberapa logis dan konsisten respons tersebut dalam konteks percakapan yang lebih luas? Apakah respons tersebut masuk akal sebagai tindak lanjut dari pesan pengguna sebelumnya?
- **Relevance (Relevansi):** Seberapa relevan respons tersebut dengan pertanyaan atau pernyataan pengguna? Apakah respons tersebut secara langsung membahas topik yang dibicarakan?
- **Fluency (Kelancaran):** Seberapa alami dan mudah dibaca respons tersebut? Apakah respons tersebut bebas dari kesalahan tata bahasa atau ejaan?
- **Factual Consistency:** Apakah informasi yang diberikan dalam respons konsisten dengan fakta yang ada dalam konteks yang diberikan (misalnya, dalam artikel yang diringkas)?

Evaluasi ini dapat dilakukan dengan meminta annotator manusia untuk memberi peringkat respons pada skala Likert (misalnya, 1-5) untuk setiap dimensi. Meskipun mahal dan memakan waktu, evaluasi manusia memberikan wawasan yang paling mendalam tentang pengalaman pengguna aktual.

4.2.2 Metrik Otomatis: BLEU, Perplexity Selain evaluasi manusia, ada beberapa metrik otomatis yang dapat memberikan perkiraan kasar tentang kualitas percakapan, meskipun mereka harus diinterpretasikan dengan hati-hati.

- **BLEU:** Seperti yang disebutkan sebelumnya, BLEU mengukur tumpang tindih n-gram dengan respons referensi. Dalam konteks percakapan, BLEU dapat digunakan jika Anda memiliki satu atau lebih respons referensi yang dianggap “benar” untuk setiap turn. Namun, BLEU cenderung tidak berkorelasi dengan evaluasi manusia dalam dialog karena ada banyak cara yang valid untuk merespons.
- **Perplexity:** Ini adalah metrik intrinsik yang mengukur seberapa “terkejut” model ketika melihat data uji. Secara matematis, ini adalah probabilitas terbalik dari set uji, yang dinormalisasi oleh jumlah kata. Perplexity yang lebih rendah umumnya menunjukkan bahwa model memiliki

pemahaman yang lebih baik tentang bahasa dan dapat memprediksi kata berikutnya dengan lebih akurat. Meskipun perplexity yang rendah tidak menjamin kualitas respons yang tinggi, perplexity yang sangat tinggi adalah indikator yang baik bahwa model tidak belajar dengan baik.

4.2.3 Evaluasi End-to-End pada Skenario Multi-Turn Evaluasi yang paling komprehensif untuk chatbot Anda adalah pengujian **end-to-end** pada skenario multi-turn yang lengkap. Ini melibatkan penciptaan serangkaian skenario uji yang meniru percakapan dunia nyata, di mana pengguna beralih antar tugas secara alami. Sebagai contoh, sebuah skenario uji mungkin terlihat seperti ini:

1. **Turn 1 (Chat):** Pengguna menyapa chatbot.
2. **Turn 2 (Summarization):** Pengguna membagikan teks berita dan bertanya untuk pendapat.
3. **Turn 3 (NER):** Pengguna bertanya siapa yang disebutkan dalam artikel tersebut.
4. **Turn 4 (Paraphrase):** Pengguna meminta chatbot untuk menjelaskan poin utama dengan cara yang berbeda.
5. **Turn 5 (Translation):** Pengguna meminta chatbot untuk menerjemahkan ringkasan tersebut ke bahasa lain.

Anda kemudian dapat menjalankan model Anda melalui skenario ini dan mengevaluasi setiap responsnya, baik secara otomatis (menggunakan metrik yang sesuai untuk tugas yang terdeteksi) maupun secara manual. Pengujian end-to-end seperti ini adalah satu-satunya cara untuk benar-benar memastikan bahwa model Anda dapat menangani kompleksitas dinamis dari percakapan multi-task implicit dan memberikan pengalaman pengguna yang mulus dan efektif.